

第1回採点課題レポート

学籍番号: cy24254

氏名: 小畑 諒英

1. 作成したプログラム (10)

```
#include <stdio.h>
#include <pthread.h>
#include <time.h>

#define COUNT 1000
#define LOOP 10000

struct timespec ts[4][COUNT];
pthread_mutex_t mutexes[3];

void busy(){
    for(int i = 0; i < LOOP; i++){
    }
}

void *fun(void *arg){
    int thn = (int)arg;
    int time_index = 0;

    for(int loop = 0; loop < 100; loop++){
        int acquired_count = 0;
        int acq[2];

        while(acquired_count < 2){
            acquired_count = 0;
            for(int i = 0; i < 3; i++){
                if(pthread_mutex_trylock(&mutexes[i]) == 0){
                    acq[acquired_count] = i;
                    acquired_count++;
                    if(acquired_count == 2) break;
                }
            }

            if(acquired_count == 1){
                pthread_mutex_unlock(&mutexes[acq[0]]);
            }
        }

        // クリティカルセクション (処理 A)
        for(int i = 0; i < 10; i++){
            busy();
            clock_gettime(CLOCK_REALTIME, &ts[thn][time_index]);
            time_index++;
        }

        pthread_mutex_unlock(&mutexes[acq[0]]);
    }
}
```

```

        pthread_mutex_unlock(&mutexes[acq[1]]);

        // 処理 B
        busy();
    }
    return NULL;
}

int main(){
    pthread_t th[4];
    struct timespec x;
    int thn, i;

    for(int m = 0; m < 3; m++){
        pthread_mutex_init(&mutexes[m], NULL);
    }

    clock_gettime(CLOCK_REALTIME, &x);
    long startt = x.tv_sec * 1000000000 + x.tv_nsec;

    for(thn = 0; thn < 4; thn++){
        pthread_create(&th[thn], NULL, fun, (void *)thn);
    }

    for(thn = 0; thn < 4; thn++){
        pthread_join(th[thn], NULL);
    }

    for(thn = 0; thn < 4; thn++){
        for(i = 0; i < COUNT; i++){
            long t = ts[thn][i].tv_sec * 1000000000 + ts[thn][i].tv_nsec;
            printf("%ld\t%d\n", t - startt, thn);
        }
    }
    return 0;
}

```

2. 設問に対する回答

問題 2：ライブロックになる時の動作について

プログラム 1B において、複数のスレッドが異なる 1 つ目のリソースを同時に確保した場合にライブロックが発生する可能性がある。例えば、スレッド A がリソース 1 を、スレッド B がリソース 2 を同時にロックしたとすると、その後に互いにもう一方のリソースを `trylock` で要求するが、すでに相手に確保されているため失敗する。

問題 3：プログラム 1C において、ヒントに書かれたような動作（手放してやり直し）にしないとデッドロックになる動作について

プログラム 1C において、「1 つ目のリソースを確保したものの、2 つ目の確保に失敗した時に持ち手を解放する」という処理を行わず、2 つ目のリソースが空くまでその場で待ち続ける設計にした場合、デッドロックが発生する。